See It, Say It, Pick It

Track 2A technical report
Team: Nightcats

1. Summary

We built a local manipulation planner that converts one workcell image and one English instruction into a checked robot plan and first-pick pixel. Apertus v1.5 8B performs language reasoning, open vision models describe and segment the scene, and deterministic constraints decide whether to plan, ask for clarification or refuse. The final public development-set score was **24.0/65**.

2. Architecture

Language. Apertus produces a constrained typed intent; a small grammar covers literal and safety-critical operation classes.

Perception. Qwen2.5-VL-7B-Instruct inventories objects, SAM 2.1 produces masks, and Grounding DINO plus SAM performs bounded recovery for unresolved source references.

Resolution. A deterministic solver checks identity, attributes, relations, uniqueness, physical feasibility and safety. Ambiguity becomes `ask_user`; unsupported, impossible or unsafe tasks become `refuse`.

Grounding. Mask geometry selects an interior `[u, v]` pixel for the first executable pick and exports only official-schema fields.

3. Use of Apertus

Model: `swiss-ai/Apertus-v1.5-8B`, revision `a411d838600baf0e3635a3daf66fb7c55fc97bb6`.
Runtime: Q8_0 GGUF served by CUDA-enabled llama.cpp b11490 on loopback with an 8192-token context. The pinned GGUF revision is `2028c3184c0fa060b4259b33ee25e836da6896e5`.
Use: local inference only, temperature 0, at most two parse attempts per source clause. Apertus proposes language slots but never invents pixels or bypasses deterministic safety checks.

4. Data

We use the organizer-provided development and held-out workcell scenes, instructions and calibration files. The challenge dataset is CC BY 4.0. We created no training set, performed no fine-tuning and package no organizer images, hidden evaluator answers or personal data in the release.

5. What was difficult for Apertus

Apertus was reliable on short literal commands but less reliable on nested relations, coordinated clauses, negation and scope. Free-form JSON also produced null-versus-empty-list errors and incomplete relation pairs. We use constrained generation, a flat typed schema, clause-level parsing, source-coverage checks and one bounded semantic repair. Outputs that silently drop a color, relation, quantity, exception or destination are rejected.

The harder end-to-end problem is often the language-perception contract. A correct noun such as gear can be labelled sun or `graspable_object` by the inventory. Targeted recovery helps, but weak visual evidence still leads to conservative clarification or refusal.

6. Evaluation

The official public development-set result was **24.0/65**: grounding ratio **0.30**, safety contribution ratio **0.45**, unsafe or impossible catch rate **0.90**, and false-alarm rate **0.50**. The system catches most unsafe cases but rejects too many valid tasks.

The held-out run produced all **40/40** required entries: 8 executable plans, 10 clarification requests and 22 refusals. Because held-out answers are private, these counts show schema and coverage only, not accuracy.

7. Limitations and next steps

Open-vocabulary perception remains the main limitation. Generic labels, duplicates, conflicting masks and visually similar containers cause missing or ambiguous references. The task representation is also incomplete for heterogeneous target unions, multiple simultaneous relations, scoped extrema and negative attributes.

Next, we would replace the manual alias layer with model-backed semantic matching, suppress duplicate and conflicting detections before relation solving, and add scale-aware evidence for grasp and container feasibility. We would then evaluate on a broader held-out-style instruction set.

8. Reproducibility and licence

The workflow runs on a Google Colab A100 40 GB runtime with Python 3.13. Models run sequentially so Apertus and the vision stack do not occupy GPU memory together. The primary entry point is `colab/Hack_Apertus_2A_Heldout_Colab.ipynb`; the README lists the equivalent staged CLI commands. Pinned weights, cache fingerprints and schema validation make each stage auditable.

Code is Apache-2.0. This report and other documentation are CC BY 4.0. Model weights are not redistributed and retain their original licences.